EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 6
AIM: Create a webpage that **retrieves, displays, and updates user data** from a MySQL database. Implement **CRUD operations (Create, Read, Update, Delete)** using PHP & MySQL. Ensure user input is **validated and securely stored** in the database.
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Dr. Yogesh Pingle

Aim: Create a webpage that **retrieves, displays, and updates user data** from a MySQL database. Implement **CRUD operations (Create, Read, Update, Delete)** using PHP & MySQL. Ensure user input is **validated and securely stored** in the database.

Theory:

Modern web applications often require interaction with databases to store, retrieve, and manage user information efficiently. This experiment focuses on developing a dynamic web application using PHP and MySQL that performs CRUD operations (Create, Read, Update, Delete) on user data. Additionally, the application ensures input validation, secure data storage, and incorporates Sessions and Cookies for user management

A. PHP and MySQL Integration

PHP (Hypertext Preprocessor) is a server-side scripting language widely used for web development. It is especially suitable for database-driven applications because it can easily interact with databases like **MySQL**.

MySQL is an open-source relational database management system (RDBMS) used to store structured data in tables. PHP communicates with MySQL using extensions such as **MySQLi** or **PDO (PHP Data Objects)** to execute SQL queries.

In this experiment, PHP scripts handle database connections, execute SQL commands, and dynamically generate HTML output based on database records.

B. CRUD Operations

CRUD represents the four fundamental database operations:

1. Create (Insert)

The **Create** operation is used to add new user records into the database. User data is collected through HTML forms and sent to the server using HTTP POST method. PHP processes the input and inserts validated data into the MySQL table using the INSERT SQL statement.

2. Read (Select)

The **Read** operation retrieves user data from the database and displays it on the webpage. PHP executes the SELECT SQL query and fetches records, which are displayed dynamically in tabular format. This operation allows users to view stored data.

3. Update

The **Update** operation modifies existing user records. The selected record is loaded into an HTML form, allowing the user to edit data. PHP then updates the record in the database using the UPDATE SQL statement.

4. Delete

The **Delete** operation removes unwanted records from the database using the DELETE SQL statement. This operation helps maintain accurate and relevant data.

C. Database Structure

A MySQL database typically contains:

- A **database** to store application data
- A **table** to store user information such as ID, name, email, and password
- A **primary key** to uniquely identify each record

Relational databases ensure data consistency and integrity through structured schema and constraints.

D. Input Validation

Input validation is essential to ensure that user-provided data is correct and safe. Validation includes:

- Checking for empty fields
- Validating email format
- Restricting input length
- Allowing only valid characters

Validation can be implemented using both **client-side validation (HTML/JavaScript)** and **server-side validation (PHP)**. Server-side validation is critical, as client-side validation can be bypassed.

E. Security Measures

SQL Injection Prevention

SQL Injection is a common attack where malicious SQL code is inserted into input fields. To prevent this:

- Prepared statements
- Parameterized queries

  are used to securely execute SQL commands.

Password Security

Passwords should never be stored in plain text. PHP provides hashing functions such as password_hash() to securely store passwords and password_verify() to authenticate users.

F. Sessions

A **session** is a server-side mechanism used to store user data temporarily across multiple pages. Sessions are commonly used for:

- User login systems
- Authentication
- Maintaining user state

In PHP, sessions are created using session_start() and data is stored in session variables. Session data is automatically destroyed when the browser is closed or the session expires.

G.Cookies

A **cookie** is a small piece of data stored on the client's browser. Cookies are used to:

- Remember user preferences
- Store login information
- Track user activity

Unlike sessions, cookies have an expiration time and remain stored even after the browser is closed. PHP uses the setcookie() function to create cookies.

H.Dynamic Web Page Behavior

The application dynamically updates content based on database operations. PHP fetches updated records after every CRUD operation, ensuring that the displayed data always reflects the current state of the database.

Algorithm/Procedure:

A. Database Creation

1. Start XAMPP/WAMP server.
2. Open phpMyAdmin.
3. Create a database named userdb.

B. CRUD Operations Flow

4. Create a table users with fields:
   - id (INT, Primary Key, Auto Increment)
   - name (VARCHAR)
   - email (VARCHAR)
   - password (VARCHAR)

   Steps:

1. User enters details in HTML form.
2. PHP validates user input.
3. Data is securely inserted into MySQL database.
4. Existing records are fetched and displayed.
5. User can update or delete records.
6. Sessions manage login state.
7. Cookies store user preference.

Code:

▪ **Db.php:**

```
<?php

$host = '127.0.0.1';
$user = 'root';
$pass = '';
$dbname = 'mdm\_exp6';

$port = 3306;

$conn = new mysqli($host, $user, $pass, $dbname, $port);

if ($conn->connect\_error) {
	die("Connection failed: " . $conn->connect\_error);
}
```

▪ **register.php**

```
<?php
require 'db.php';
$message = '';

if ($\_SERVER\["REQUEST\_METHOD"] == "POST") {
	$username = trim($\_POST\['username']);
	$email = trim($\_POST\['email']);
	$password = $\_POST\['password'];

	if (empty($username) || empty($email) || empty($password)) {
		$message = "All fields are required!";
	} elseif (!filter\_var($email, FILTER\_VALIDATE\_EMAIL)) {
	$message = "Invalid email format!";
	} else {
		$hashed\_password = password\_hash($password, PASSWORD\_DEFAULT);

		$stmt = $conn->prepare("INSERT INTO users (username, email, password)
		VALUES (?, ?, ?)");
		$stmt->bind\_param("sss", $username, $email, $hashed\_password);

		if ($stmt->execute()) {
			$message = "Registration successful! <a href='display.php'>View
			Users</a>";
		} else {
			$message = "Error: " . $stmt->error;
		}

		$stmt->close();
```

```
        }
}
$conn->close();
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>User Registration</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 50px; }
        .container { max-width: 400px; margin: auto; padding: 20px; border: 1px solid #ccc;
border-radius: 5px; }
        input[type="text"], input[type="email"], input[type="password"] { width: 100%;
padding: 10px; margin: 10px 0; box-sizing: border-box; }
        button { background-color: #28a745; color: white; padding: 10px 15px; border: none;
cursor: pointer; width: 100%; }
        button:hover { background-color: #218838; }
        .message { color: red; margin-bottom: 15px; }
    </style>
</head>
<body>
    <div class="container">
        <h2>User Registration System</h2>
        <?php if($message != '') echo "<div class='message'>$message</div>"; ?>
        <form action="register.php" method="POST">
            <label>Username</label>
            <input type="text" name="username" required>

            <label>Email</label>
            <input type="email" name="email" required>

            <label>Password</label>
            <input type="password" name="password" required>

            <button type="submit">Register</button>
        </form>
    </div>
</body>
</html>
```

▪ **display.php**

```
<?php
require 'db.php';

$sql = "SELECT id, username, email, created_at FROM users ORDER BY id DESC";
$result = $conn->query($sql);
?>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Registered Users</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 50px; }
        .controls { display: flex; justify-content: space-between; margin-bottom: 20px; }
        input[type="text"] { padding: 8px; width: 300px; border: 1px solid #ccc; border-radius:
4px;}
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 12px; border: 1px solid #ddd; text-align: left; }
        th { background-color: #f4f4f4; }
        .nav-link { text-decoration: none; color: #007bff; }
        .btn-edit { color: white; background-color: #ffc107; padding: 5px 10px; text-decoration:
none; border-radius: 3px; }
        .btn-delete { color: white; background-color: #dc3545; padding: 5px 10px; text-
decoration: none; border-radius: 3px; }
    </style>
</head>
<body>
    <h2>Registered Users Dashboard</h2>

    <div class="controls">
        <a href="register.php" class="nav-link">&larr; Add New User</a>
        <input type="text" id="searchBox" placeholder="Search by username or email...">
    </div>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Username</th>
                <th>Email</th>
                <th>Registration Date</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody id="userData">
            <?php
            if ($result->num_rows > 0) {
                while($row = $result->fetch_assoc()) {
                    echo "<tr>
                            <td>{$row['id']}</td>
                            <td>" . htmlspecialchars($row['username']) . "</td>
                            <td>" . htmlspecialchars($row['email']) . "</td>
                            <td>{$row['created_at']}</td>
                            <td>
                                <a href='edit.php?id={$row['id']}' class='btn-edit'>Edit</a>
```

```
                <a href='delete.php?id={$row['id']}' class='btn-delete' onclick='return
            confirm(\"Are you sure you want to delete this user?\");'>Delete</a>
                </td>
               </tr>";
            }
        } else {
            echo "<tr><td colspan='5'>No users found.</td></tr>";
        }
        ?>
    </tbody>
  </table>

  <script>
    document.getElementById('searchBox').addEventListener('keyup', function() {
        let query = this.value;

        // Use the Fetch API to send data to our search backend
        fetch(`search.php?q=${encodeURIComponent(query)}`)
            .then(response => response.text())
            .then(data => {
                // Update the table body with the results
                document.getElementById('userData').innerHTML = data;
            })
            .catch(error => console.error('Error:', error));
    });
  </script>
</body>
</html>
```

▪ **edit.php**

```
<?php
require 'db.php';

if (!isset($_GET['id'])) {
    header("Location: display.php");
    exit();
}

$id = $_GET['id'];
$message = '';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = trim($_POST['username']);
    $email = trim($_POST['email']);

    if (empty($username) || empty($email)) {
        $message = "Username and Email are required.";
    } else {
        $stmt = $conn->prepare("UPDATE users SET username=?, email=? WHERE id=?");
```

```
        $stmt->bind_param("ssi", $username, $email, $id);

        if ($stmt->execute()) {
            header("Location: display.php"); // Go back to display on success
            exit();
        } else {
            $message = "Error updating record: " . $conn->error;
        }
        $stmt->close();
    }
}

// Fetch current user data to pre-fill the form
$stmt = $conn->prepare("SELECT username, email FROM users WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc();

if (!$user) {
    die("User not found!");
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Edit User</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 50px; }
        .container { max-width: 400px; margin: auto; padding: 20px; border: 1px solid #ccc;
border-radius: 5px; }
        input[type="text"], input[type="email"] { width: 100%; padding: 10px; margin: 10px 0;
box-sizing: border-box; }
        button { background-color: #ffc107; color: black; padding: 10px 15px; border: none;
cursor: pointer; width: 100%; }
        .message { color: red; margin-bottom: 15px; }
    </style>
</head>
<body>
    <div class="container">
        <h2>Edit User</h2>
        <?php if($message != '') echo "<div class='message'>$message</div>"; ?>

        <form action="" method="POST">
            <label>Username</label>
            <input type="text" name="username" value="<?php echo
htmlspecialchars($user['username']); ?>" required>
```

```
            <label>Email</label>
            <input type="email" name="email" value="<?php echo
htmlspecialchars($user['email']); ?>" required>

            <button type="submit">Update User</button>
        </form>
        <br>
        <a href="display.php">Cancel</a>
    </div>
</body>
</html>
```

▪ **delete.php**

```
<?php
require 'db.php';

if (isset($_GET['id'])) {
    $id = $_GET['id'];


$stmt = $conn->prepare("DELETE FROM users WHERE id = ?");
    $stmt->bind_param("i", $id);

    if ($stmt->execute()) {
        // Successfully deleted, go back to display page
        header("Location: display.php");
    } else {
        echo "Error deleting record: " . $conn->error;
    }

    $stmt->close();
}
$conn->close();
?>
```

▪ **search.php**

```
<?php
require 'db.php';

$query = isset($_GET['q']) ? $_GET['q'] : '';

$sql = "SELECT id, username, email, created_at FROM users WHERE username LIKE ? OR
email LIKE ?";
$stmt = $conn->prepare($sql);

$searchTerm = "%" . $query . "%";
$stmt->bind_param("ss", $searchTerm, $searchTerm);
$stmt->execute();
```

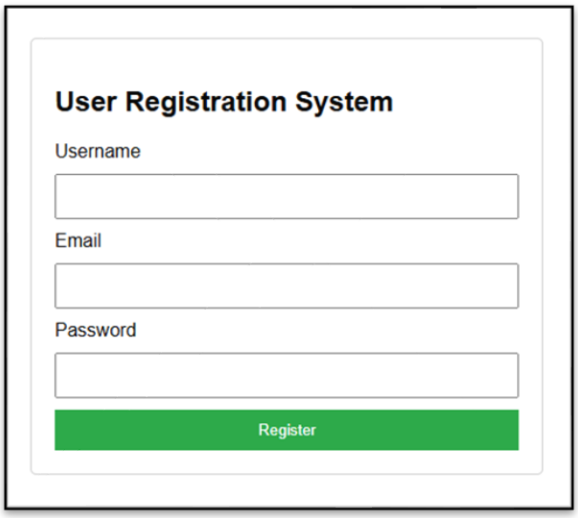
```
$result = $stmt->get_result();

if ($result->num_rows > 0) {
   while($row = $result->fetch_assoc()) {
      echo "<tr>
            <td>{$row['id']}</td>
            <td>" . htmlspecialchars($row['username']) . "</td>
            <td>" . htmlspecialchars($row['email']) . "</td>
            <td>{$row['created_at']}</td>
            <td>
               <a href='edit.php?id={$row['id']}' class='btn-edit'>Edit</a>
               <a href='delete.php?id={$row['id']}' class='btn-delete' onclick='return
confirm(\"Are you sure?\");'>Delete</a>
            </td>
         </tr>";
   }
} else {
   echo "<tr><td colspan='5'>No matching users found.</td></tr>";
}

$stmt->close();
$conn->close();
?>
```

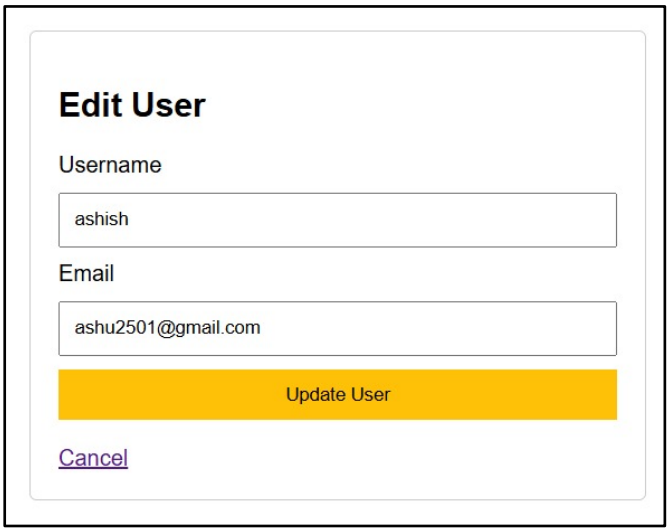
Output:

First page: Here the user enters his username, email id and password. After entering the details, the user will get register.

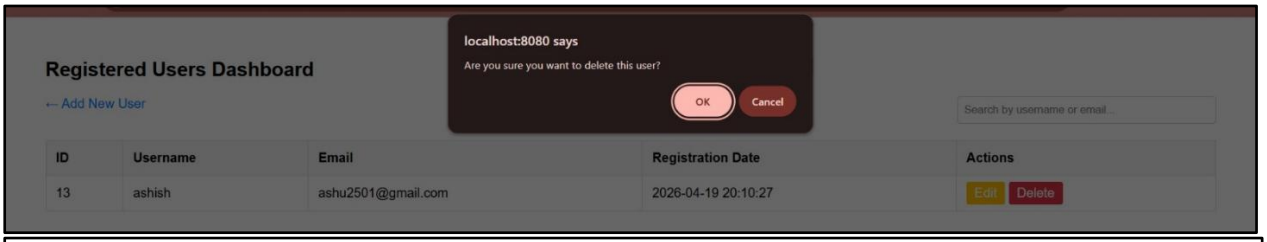


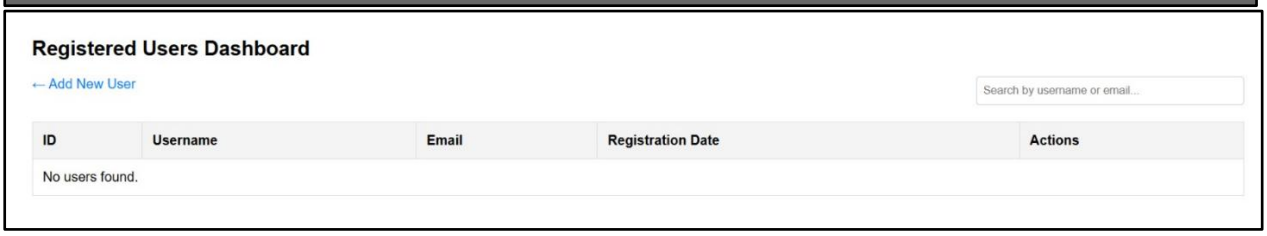
User List page: Here all the users will be listed, with the details that he entered during the registration.

Editing page: Here we will be able to change his username and email.



Delete page: We can delete the user, by clicking the "delete button"





Conclusion

Thus, a dynamic database-driven web application was successfully developed using PHP and MySQL. CRUD operations were implemented with secure data handling, input validation, and session & cookie management. This experiment demonstrates real-world server-side web development concepts.

Question:

1. **What is the role of PHP in database connectivity?**

Ans:
Definition: PHP acts as a server-side scripting language that connects web applications to databases (like MySQL).
Key Points:

- Establishes connection using functions like `mysqli_connect()` or PDO.
- Executes SQL queries (INSERT, UPDATE, SELECT, DELETE).
- Fetches and displays dynamic data on web pages.
- Provides error handling and secure communication with the database.

2. **Difference between session and cookie?**

Ans:

Aspect	Session	Cookie
Definition	Server-side storage of user data	Client-side storage of user data in browser
Location	Stored on server	Stored on user's computer
Security	More secure (not directly accessible by user)	Less secure (user can view/modify cookies)
Lifetime	Ends when browser is closed or session expires	Can persist for days/weeks until expiry date
Use Case	Login authentication, temporary data	Remembering preferences, auto-login

3. **How are passwords secured in PHP?**

Ans:
Definition: PHP provides built-in functions to secure passwords before storing them in databases.
Key Points:

- Use password_hash() → generates a secure hash of the password.
- Use password_verify() → checks entered password against stored hash.
- Prevents storing plain-text passwords.
- Protects against attacks like brute force and SQL injection.